

- 以下是 SecurityAccess.c 中 UDS 0x27 安全解锁的完整流程。

整体架构

实现了 UDS 0x27 服务的两个安全等级：

- **Level 1 (0x01)**: 常规诊断安全访问
- **Level 3 (0x03)**: 更高安全等级

存在两套算法体系：

- **旧车型** (E001/E007/E009/E202/C801, 非P301系)：自定义位运算算法, 4 字节 seed/key
- **新车型** (P301/P201/P567/C206_10/E001_10/E202_10/C801)：国密 SM4 算法, 16 字节 seed/key

1. 初始化阶段

SecurityAccess_CarModeSetParams() (第61行) 根据 Car_Model 设置 seed 和 key 的长度：

车型	seed 长度	key 长度	算法
P301/P201/P567/C206_10/E001_10/E202_10/C801	16 字节 (FAW_P301_SEEDLEN)	16 字节 (FAW_P301_KEYLEN)	SM4
其他 (E001/E007/E009/E202 旧款等)	4 字节 (FAW_NOTP301_SEEDLEN)	4 字节 (FAW_NOTP301_KEYLEN)	位运算

同时会修改 Dcm_Lcfg 中相关数组, 使 DCM 协议栈的请求/响应长度与车型匹配。

2. 上电延时 NRC37 检查

SecurityAccess_CheckPoweronNRC37() (第167行), 每 10ms 周期调用一次：

- 上电 10 个 tick 内, 若 AttCnt >= 1, 启动检查标志位
- 上电 60 秒 (6000 tick) 内, 对应标志位为 TRUE, 此时 GetSeed 请求会被拒绝, 返回 NRC 0x37 (requiredTimeDelayNotExpired)
- 超过 60 秒后, 标志位恢复 FALSE, 正常响应

这是一种防暴力破解机制。

3. 标准 27 服务解锁流程

3.1 Step 1: Tester 请求 Seed (27 01 / 27 03)

DCM 协议栈收到请求后, 回调对应函数：

- L1: SecurityAccess_Unlocked_0x01_GetSeed() (第442行)
- L3: SecurityAccess_Unlocked_0x03_GetSeed() (第614行)

两者逻辑完全对称：

NRC37 检查 - 通过?
└─ NO -> 返回 0x37
└─ YES -> Static_Seed_flag == FALSE?
└─ YES -> 调用 SecurityAccess_GetSeed() 生成随机种子
└─ 存入 g_nvm_for_sercurityLX[2..] 备份
└─ 设置 Static_Seed_flag = TRUE
└─ 同时存入 sP301LXSeed (SM4车型) 或 sSeedLX(位运算车型)
└─ NO -> 从 g_nvm_for_sercurityLX[2..] 取出上次的种子复用

关键点: Static_Seed_flag 机制保证同一安全等级的整个解锁过程中, seed 保持不变。只有 key 校验通过后会才会清除该标志位。

SecurityAccess_GetSeed() (第219行) 的随机数生成方式: 用 CLOCK_MONOTONIC 纳秒时间戳作为种子, 每字节递增一次时间戳, 调用 srand() + rand() 产生 1~255 的随机字节。

3.2 Step 2: Tester 发送 Key (27 02 / 27 04)

DCM 协议栈收到 key 后, 回调对应函数：

- L1: SecurityAccess_Unlocked_0x01_CompareKey() (第336行)
- L3: SecurityAccess_Unlocked_0x03_CompareKey() (第514行)

两者逻辑完全对称：

生成 ECU 侧期望的 key

SM4 车型 (P301/P201/P567/C206_10/E001_10/E202_10/C801)：调用 SecurityAccess_SM4Encrypt() (第125行), 使用 cyber 加密库的 SM4-ECB 模式, 以预设的 cipher 密钥对之前存储的 seed 进行加密, 生成 16 字节 key。每个车型有不同的 SM4 密钥常量。

位运算车型 (旧款 E001/E007/E009/E202 等)：

- L1 -> SecurityAccess_SeedToKeyL1() (第245行)：循环 35 次, 根据最高位是否为 1 决定左移后异或 mask 还是仅左移
- L3 -> SecurityAccess_SeedToKeyL3() (第276行)：更复杂的算法, 涉及从 mask 中提取参数决定迭代次数、字节交换等

比较 key

调用 SecurityAccess_CompareKey() (第95行) 逐字节比较：

- **匹配成功** -> 返回 DCM_E_POSITIVERESPONSE (正响应), 清除 Static_Seed_flag, 上传 [Normal] 安全日志
- **匹配失败** -> 返回 DCM_E_INVALIDKEY (无效 key), DCM 协议栈会递增 AttCnt, 清除 Static_Seed_flag, 同时：
 - 打印 DEBUG_INFO 日志
 - 将错误详情写入 error_info_data 全局缓冲区并设置 error_info_flag = true (供上传到云端)
 - 上传 [Moderate] 安全日志

3.3 AttCnt (尝试计数器) 管理

DCM 协议栈在 key 校验失败后, 会通过以下回调读写 AttCnt：

回调	功能
SecurityAccess_Unlocked_0x01_GetAttemptCounter()	读取 g_nvm_for_sercurityl1[0]

SecurityAccess_Unlocked_0x01_SetAttemptCounter()	写入 g_nvm_for_securityl1[0]
SecurityAccess_Unlocked_0x03_GetAttemptCounter()	读取 g_nvm_for_securityl3[0]
SecurityAccess_Unlocked_0x03_SetAttemptCounter()	写入 g_nvm_for_securityl3[0]

AttCnt 存储在 NVM 数组的第 0 字节，掉电不丢失。连续失败达到阈值后 DCM 协议栈会进入延时惩罚。

4. 关键数据结构

g_nvm_for_securityl1[20]:

- [0] = AttCnt (尝试计数器, NVM持久化)
- [1] = Static_Seed_flag (是否复用种子)
- [2..17] = Seed 备份 (最多 16 字节)

g_nvm_for_securityl3[20]: 同上, Level 3 独立维护

sP301L1Seed[16] / sP301L3Seed[16] → SM4 车型的明文种子
sSeedL1 / sSeedL3 → 位运算车型的 32 位种子

5. 流程图总结



```
</ SecurityAccess.c
@src/dsar_fw/product/faw/oem_feature/diagnosis/SecurityAccess/SecurityAccess.c 帮我分析下mcu域的2701安全解锁，c100-10项目需要把seed和key的长度改为16，应该如何修改，你帮我理下这个dcm_lcfg这些静态配置的链路，怎么连接到oem的这个函数上的
```

- Thinking >
- Let me trace the full configuration chain. I'll read the relevant files in parallel.

Read SecurityAccess.h, SecurityAccess_If.h

Bash Find Dcm_Lcfg files in dsar_fw

```
IN find /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw -name "Dcm_Lcfg*" -type f 2>/dev/null

OUT /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_p301_10/microsar_config/Appl/GenData/Dcm_Lcfg.c
/home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_p301_10/microsar_config/Appl/GenData/Dcm_Lcfg.h
/home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_p301/microsar_config/Appl/GenData/Dcm_Lcfg.c
```

Bash Find SM4 header for MCU side

```
IN find /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw -name "SecurityAccess_SM4*" -type f 2>/dev/null

OUT /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/SecurityAccess/SecurityAccess_SM4.c
/home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/SecurityAccess/SecurityAccess_SM4.h
```

- Thinking >
- Read Dcm_Lcfg.h
- Read Dcm_Lcfg.c
- Read Dcm_Lcfg.h
- Read SecurityAccess_SM4.h
- Thinking >
- Let me read the C801 Dcm_Lcfg for the 27 service config and the C100-10 Dcm_Lcfg.h for related definitions.

Bash Extract 0x27 related config from C801 Dcm_Lcfg.c

```
IN grep -n "Svc27\|SecurityAccess\|SecLevel\|SeedRes\|SubFunc" /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_c801/microsar_config/Appl/GenData/Dcm_Lcfg.c

OUT 176:CONST(uint8, DCM_CONST) Dcm_CfgSvc10SubFuncLookupTable[4]=
184:CONST(uint8, DCM_CONST) Dcm_CfgSvc11SubFuncLookupTable[3]=
191:CONST(uint8, DCM_CONST) Dcm_CfgSvc19SubFuncLookupTable[8]=
```

Bash Extract 0x27 related config from C100-10 Dcm_Lcfg.h

```
IN grep -n "Svc27\|SecurityAccess\|SecLevel\|SeedRes\|SubFunc\|SecLv1\|Dcm_CfgSvc27" /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_c100_10_f6m/microsar_config/Appl/GenDat

OUT 82:# define Dcm_CfgWrapSvc11SubFuncInfo(index) (Dcm_CfgSvc11SubFuncInfo[(index)]) /* PRQA S 3453 */ /* MD_MSR_FctLikeMacro */
83:# define Dcm_CfgWrapSvc19SubFuncInfo(index) (Dcm_CfgSvc19SubFuncInfo[(index)]) /* PRQA S 3453 */ /* MD_MSR_FctLikeMacro */
```

```
83:# define Dcm_CfgWrapSvc19SubFuncInfo(index) (&Dcm_CfgSvc19SubFuncInfo[(index)])
84:# define Dcm_CfgWrapSvc28SubFuncInfo(index) (&Dcm_CfgSvc28SubFuncInfo[(index)])
/* PRQA S 3453 */ /* MD_MSR_FctLikeMacro */
```

● **Bash** Extract 0x27 related config from C801 Dcm_Lcfg.h

```
IN grep -n "Svc27\|SecurityAccess\|SecLevel\|SeedRes\|SubFunc\|SecLvl\|Dcm_CfgSvc27" /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_c801/microsar_config/Appl/GenData/Dcm_L

OUT 79:# define Dcm_CfgWrapSvc11SubFuncInfo(index) (&Dcm_CfgSvc11SubFuncInfo[(index)]) /* PRQA S 3453 */ /* MD_MSR_FctLikeMacro */
80:# define Dcm_CfgWrapSvc19SubFuncInfo(index) (&Dcm_CfgSvc19SubFuncInfo[(index)]) /* PRQA S 3453 */ /* MD_MSR_FctLikeMacro */
81:# define Dcm_CfgWrapSvc28SubFuncInfo(index) (&Dcm_CfgSvc28SubFuncInfo[(index)]) /* PRQA S 3453 */ /* MD_MSR_FctLikeMacro */
```

● Thinking >

● **Read** Dcm_Lcfg.c (lines 621-645)

● **Bash** Search for C100-10 car model define

```
IN grep -rn "FAW_P301DIAG\|FAW_C100\|CARMODEL.*C100\|C100_10\|c100" /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_c100_10_j6m/ --include="*.h" --include="*.c" -l 2>/dev/n

OUT /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_c100_10_j6m/config/product_configs.h
```

● **Bash** Search for car_model related code in c100-10

```
IN grep -rn "get_car_model\|car_model\|CAR_MODEL\|CARMODEL" /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_c100_10_j6m/ --include="*.h" --include="*.c" -l 2>/dev/null | he

OUT /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/fawhq_c100_10_j6m/proxy/SensorGroup/mradar_proxy.h
```

● Thinking >

● **Read** product_configs.h

● **Bash** Search for FAW_P301DIAG and C100 car model across all faw projects

```
IN grep -rn "FAW_P301DIAG\|C100_10_CARMODEL\|C100.*CARMODEL" /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/ --include="*.h" --include="*.c" 2>/dev/null | head -20

OUT /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/SecurityAccess/SecurityAccess.h:36:#ifdef FAW_P301DIAG
```

● Thinking >

● Now I have all the information needed. Let me also quickly check if C100-10 has its own car model define or uses an existing one.

● **Bash** Search for C100 car model define in dsar_fw

```
IN grep -rn "FAW_C100\|C100.*CARMODEL\|CARMODEL.*33B" /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/ --include="*.h" --include="*.c" 2>/dev/null | head -10

OUT /home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/SecurityAccess/SecurityAccess.h:29:#define FAW_P567_CARMODEL (33873u)
/home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/SecurityAccess/SecurityAccess.h:30:#define FAW_C206_10_CARMODEL (33876u)
/home/c-ziming.jiang/catkin_ws/src/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/SecurityAccess/SecurityAccess.h:31:#define FAW_E001_10_CARMODEL (33885u)
```

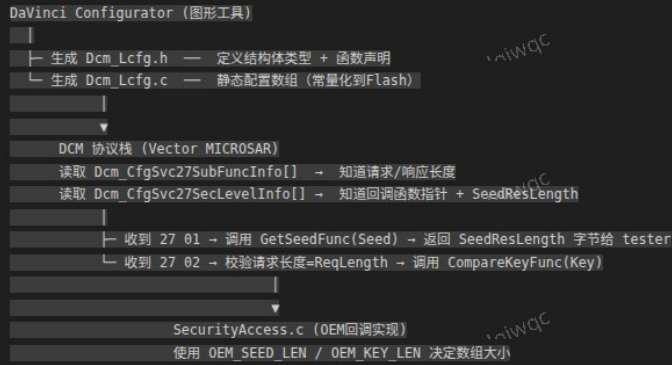
● **Read** SecurityAccess_SM4.c

● Thinking >

● Now I have the complete picture. Here is the analysis.

MCU 域 2701 安全解锁 — 配置链路 & 修改分析

一、配置链路总览



二、C100-10 当前状态 vs 目标 (C801 参考)

配置项	C100-10 (当前)	C801 (目标参考)
Dcm_Lcfg.h 函数声明	SecurityAccess_UnlockL1_*	SecurityAccess_Unlocked_0x01_*
Dcm_Lcfg.c SF 0x02 ReqLength	5 (key=4 bytes)	17 (key=16 bytes)
Dcm_Lcfg.c SeedResLength	4	16
Dcm_Lcfg.c SecLevelInfo 结构	仅有 GetSeed + CompareKey	含 GetAttemptCntnr + SetAttemptCntnr
SecurityAccess.h OEM_SEED_LEN	4	16
SecurityAccess.h OEM_KEY_LEN	4	16
SecurityAccess.c 加密算法	位运算 SecurityAccess_GetKey()	SM4 SecurityAccess_SM4Encrypt()

三、关键代码对照

C100-10 Dcm_Lcfg.c 的 0x27 配置 (行690-699):

```
// 子功能属性
Dcm_CfgSvc27SubFuncInfo[2] = {
    { 1u }, // SF 0x01: ReqLength=1
    { 5u }, // SF 0x02: ReqLength=5 → key只有4字节
};
// 安全等级属性
Dcm_CfgSvc27SecLevelInfo[1] = {
    { SecurityAccess_UnlockL1_GetSeed, // ~ 旧函数名
      SecurityAccess_UnlockL1_CompareKey,
      4u, 0u } // SeedResLength=4, 无AttemptCntnr回调
};
```

C801 Dcm_Lcfg.c 的 0x27 配置 (行624-633):

```
Dcm_CfgSvc27SubFuncInfo[2] = {
    { 1u }, // SF 0x01: ReqLength=1
    { 17u }, // SF 0x02: ReqLength=17 → key=16字节
};
Dcm_CfgSvc27SecLevelInfo[1] = {
    { SecurityAccess_Unlocked_0x01_GetSeed,
      SecurityAccess_Unlocked_0x01_CompareKey,
      SecurityAccess_Unlocked_0x01_GetAttemptCounter, // 新增
      SecurityAccess_Unlocked_0x01_SetAttemptCounter, // 新增
      16u, 0u } // SeedResLength=16
};
```

SecurityAccess.h (行36-44) 控制 seed/key 长度:

```
#ifdef FAW_P301DIAG
#define OEM_SEED_LEN (16U)
#define OEM_KEY_LEN (16U)
#else
#define OEM_SEED_LEN (4U) // ~ C100-10 当前走这个分支
#define OEM_KEY_LEN (4U)
#endif
```

四、C100-10 需要修改的文件清单

1. DaVinci 重新生成 Dcm_Lcfg.h / Dcm_Lcfg.c (核心)

需要在 DaVinci 工具中将 0x27 服务的以下参数修改:

- SF 0x02 的 ReqLength: 5 → 17
- SecLevel 的 SeedResLength: 4 → 16
- 加上 GetAttemptCounter / SetAttemptCounter 回调
- 函数名从 SecurityAccess_UnlockL1 * 更新为 SecurityAccess_Unlocked_0x01 * (匹配当前 SecurityAccess.c 的实现)

2. SecurityAccess.h — 打开 16 字节宏开关

两种方式:

- 方式A: 在 C100-10 的编译选项中定义 FAW_P301DIAG
- 方式B: 修改条件编译, 加入新车型 (推荐, 见下方)

3. SecurityAccess.c — 加入 SM4 加密分支

需要三处修改:

a) 定义 C100-10 的车型常量 (SecurityAccess.h):

```
#define FAW_C100_10_CARMODEL (33877u) // 需确认具体值
```

b) GetSeed 函数 (SecurityAccess.c 行161): 把 C100-10 加入 SM4 分支, 让 seed 存入 sP301L1Seed[16]:

```
if((get_car_model() == FAW_P301_CARMODEL) || ... || (get_car_model() == FAW_C100_10_CARMODEL))
{
    memcpy(sP301L1Seed, Seed, OEM_SEED_LEN);
}
```

c) CompareKey 函数 (SecurityAccess.c 行240): 增加 C100-10 的 SM4 密钥常量并走 SM4 加密路径:

```
else if(get_car_model() == FAW_C100_10_CARMODEL)
{
    const uint32_t C100_10_L1Cipher[4] = {0x????????, 0x????????, 0x????????, 0x????????};
    SecurityAccess_SM4Encrypt(sP301L1Seed, OEM_SEED_LEN, genKeyTemp, OEM_KEY_LEN, C100_10_L1Cipher);
}
```

SM4 密钥 (cipher) 需要从项目侧 (安全团队) 获取。

4. SecurityAccess_SM4.c / .h — 已存在, 无需修改

SM4 实现文件已在 SecurityAccess_SM4.c 中完整实现, 直接 #include "SecurityAccess_SM4.h" 即可使用。

五、修改总结

```
DaVinci 重生成 Dcm_Lcfg
├─ ReqLength: 5~17, SeedResLength: 4~16
├─ 函数名: 解锁 ~ Get/SetAttemptCntnr 回调
```

```
└─ 函数名对齐 + 加 AttemptCntIf 回调

SecurityAccess.h
├─ #define FAW_C100_10_CARMODEL (33877u)
└─ OEM_SEED_LEN/OEM_KEY_LEN 条件编译改造

SecurityAccess.c
├─ GetSeed: 加入 C100-10 → 走 SM4 分支存 seed
├─ CompareKey: 加入 C100-10 → 走 SM4 加密 + 对应 cipher
└─ #include "SecurityAccess_SM4.h"
```

核心要点: `Dcm_Lcfg` 是 `Vector DCM` 协议栈的静态配置入口, 它告诉 DCM 栈“这个安全等级的 `seed` 有多长、`key` 有多长、回调函数是谁”。DCM 栈在收到 27 请求时, 按配置长度做长度校验, 然后回调到 `SecurityAccess.c` 的 OEM 函数。OEM 函数内部再根据 `OEM_SEED_LEN / OEM_KEY_LEN` 和车型判断决定用哪种算法。两个层面必须对齐。

在安全访问中L27和L1是什么区别

- Thinking >
- 在代码语境里, “27” 和 “L1” 是不同维度的概念:

27 — UDS 服务号

0x27 是 UDS 诊断协议中「安全访问」(SecurityAccess) 服务的 ID。它定义了一套请求-响应的交互流程:

Tester	ECU
--- 27 01 (RequestSeed) -->	步骤1: 请求种子
<-- 67 01 (Seed) -----	步骤2: 返回种子
--- 27 02 (SendKey) ----->	步骤3: 发送算好的Key
<-- 67 02 (Unlock OK) ----	步骤4: 解锁成功

L1 (Level 1) — 安全等级

同一套 27 服务下可以有多个安全等级, 每个等级有独立的 `seed/key` 算法和种子:

等级	子功能码	典型用途
L1 (Level 1)	27 01 / 27 02	常规诊断操作 (读 DID、清 DTC 等)
L3 (Level 3)	27 03 / 27 04	敏感操作 (写 VIN、刷写、标定等)

代码中的对应关系

从你打开的两个文件看:

- `SecurityAccess_Unlocked_0x01_GetSeed / CompareKey` → 处理 **L1** (子功能 0x01/0x02)
- `SecurityAccess_Unlocked_0x03_GetSeed / CompareKey` → 处理 **L3** (子功能 0x03/0x04)

L1 和 L3 各自独立维护自己的 `g_securitylevel1[20] / g_securitylevel3[20]` (种子、尝试计数器都是独立的), 算法 `mask` 也不同 (`SECURITY_L1_MASK` vs `SECURITY_L3_MASK`)。

一句话

27 是服务框架 (协议怎么交互), L1/L3 是安全等级 (交互时用哪套密钥算法和种子)。你说的 “2701” 就是 27 服务下的 Level 1 RequestSeed 请求。

abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd你帮我看下这个commit更改了什么

- Thought for 9s >

- Bash Show commit changes summary

```
IN git show abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd --stat

OUT commit abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd
Author: c-ziming.jiang <c-ziming.jiang@zyt.com>
Date: Tue May 19 16:30:19 2026 +0800
```

- Thought for 3s >

- Bash Show commit diff for Dcm_Lcfg.c

```
IN git show abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd --no-stat -p -- Dcm_Lcfg.c | head -500

OUT fatal: 未识别的参数: --no-stat
```

- Bash Show commit diff for Dcm_Lcfg.h

```
IN git show abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd --no-stat -p -- Dcm_Lcfg.h | head -600

OUT fatal: 未识别的参数: --no-stat
```

- Thought for 2s >

- Bash Show diff for Dcm_Lcfg.c

```
IN git diff abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd^...abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd -- Dcm_Lcfg.c

OUT (Bash completed with no output)
```

- Bash Show diff for Dcm_Lcfg.h

```
IN git diff abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd^...abcf8b1b58d3d052ea599ec9dbbf0cb7b1d16fd -- Dcm_Lcfg.h
```

```
OUT (Bash completed with no output)
```

Thought for 2s >

Bash

Show full commit changes

```
IN git show abcfc8b1b58d3d052ea599ec9dbbf0cb7b1d16fd -p 2>&1 | head -800

OUT <persisted-output>
Output too large (50.5KB). Full output saved to: /home/c-ziming.jiang/.claude/projects/-home-c-ziming-jiang-catkin-ws-src-dsar-hq/80d94ac3-36ec-475a-8e9b-029abaa0fe2e/tool-results/beptdf6sq.txt
```

Thinking >

让我查看关于 0x27 安全访问的关键变更。

Bash

Filter diff for 0x27 and SecurityAccess related changes

```
IN grep -n "27\|SecurityAccess\|SeedRes\|SecLvl\|SecLevel\|SubFuncInfo\[\" /home/c-ziming.jiang/.claude/projects/-home-c-ziming-jiang-catkin-ws-src-dsar-hq/80d94ac3-36ec-475a-8e9b-029abaa0fe2e/tool-

OUT 32: @@ -271,22 +271,6 @@ CONST(uint16, DCM_CONST) Dcm_CfgDidMgrDidLookUpTable[38]=
84:      , { 1u, 1u, 1u, 27u, 0u, 0x03u} /* DID: 0x5999 */
110:      , { 2u, 27u, 0x01u} /* DID: 0x5999 */
```

Thought for 3s >

Read

beptdf6sq.txt (lines 241-270)

Read

beptdf6sq.txt (lines 218-247)

Thinking >

这个 commit 修改的是 C801 项目的 Dcm_Lcfg.c / Dcm_Lcfg.h，提交信息是「替换p301_dcmclfg文件」，本质是用 P301 的新版 DaVinci 生成的配置文件覆盖了旧的。

与 27 服务安全访问直接相关的变更

1. SF 0x02 的 ReqLength: 5 → 17 (key: 4字节 → 16字节)

```
Dcm_CfgSvc27SubFuncInfo[2] = {
    { 1u }, // SF 0x01 不变
-   { 5u }, // SF 0x02: 旧 ReqLength=5 → key=4字节
+   { 17u }, // SF 0x02: 新 ReqLength=17 → key=16字节
};
```

2. SeedResLength: 4 → 16 (seed: 4字节 → 16字节)

```
Dcm_CfgSvc27SecLevelInfo[1] = {
    SecurityAccess_Unlocked_0x01_GetSeed,
    SecurityAccess_Unlocked_0x01_CompareKey,
    SecurityAccess_Unlocked_0x01_GetAttemptCounter,
    SecurityAccess_Unlocked_0x01_SetAttemptCounter,
-   4u, 0u } // 旧: seed=4字节
+   16u, 0u } // 新: seed=16字节
};
```

3. 0x27 服务的安全访问权限收紧: 0x03 → 0x02

```
- , { Dcm_Service27Processor, 0x03u, ... } // 旧: 扩展会话+安全解锁后才能用
+ , { Dcm_Service27Processor, 0x02u, ... } // 新: 仅需扩展会话
```

4. 0x27 的执行前置条件: 0u → 3u

```
- , 0u /* SID: 0x27 */ // 旧: 无前置条件(DefaultModeRule)
+ , 3u /* SID: 0x27 */ // 新: 引用更严格的mode rule
```

其他变更 (非27相关)

- DID 列表精简: 删除了 F187, F189, F18C, F190, F191, F193, F195, F198, F199, F19D, F1A0-F1A3, F1A7, F1A9 共 16 个 DID (DID 总数从 37 → 21)
- 0x19 / 0x85 服务的安全权限和前置条件也做了类似调整
- Dcm_Lcfg.h 中删除了约 549 行不再需要的函数声明 (对应删除的 DID)

一句话总结

这个 commit 把 C801 的 Dcm_Lcfg 从旧版 (4字节 seed/key、老车型配置) 替换为 P301 风格的新版 (16字节 seed/key、SM4 加密)，与你之前问的 “C100-10 改成 16 字节” 是同一类改动——C801 已经改完了，C100-10 还需要做。

Dcm_CfgDiagSvcIdExecPrecondTable Dcm_CfgDiagServiceInfo这些配置都变更了正常吗

Thought for 29s >

正常，这是因为 Dcm_Lcfg.c / Dcm_Lcfg.h 是 DaVinci Configurator 全量生成 的文件，不是手写的。当从 P301 项目整份替换过来时，两个 DaVinci 项目之间所有的配置差异都会体现出来。但这里有些变更更需要确认是否真的适合 C801:

需要关注的变更

需要关注的变更

1. 0x27 服务的安全等级: 0x03 -- 0x02

```
-{ Dcm_Service27Processor, 0x03u, ...} // 旧: 需要解锁 L3 才能用 0x27 服务
+{ Dcm_Service27Processor, 0x02u, ...} // 新: 解锁 L1 即可用
```

这里语义变了。如果 C801 只需要 L1 安全解锁就能访问 0x27 服务, 那 0x02 是对的; 如果原设计是要求更高的安全等级, 那需要改回 0x03。

2. 0x27 的执行前置条件: 0u -- 3u

```
- 0u /* SID: 0x27 */ // DefaultModeRule (永远允许)
+ 3u /* SID: 0x27 */ // 引用更严格的 mode rule
```

索引从 0 变为 3, 意味着 DaVinci 中新增了一条 mode rule (编号 3), 对应 Dcm_CfgStatePreconditions[] 和 Dcm_CfgModeMgrRules[] 数组也会多出条目。这是正常的——P301 的配置对 27 服务施加了更严格的执行条件 (比如必须在扩展会话+特定模式下才能用)。

3. Dcm_Lcfg.h 删除了 549 行函数声明

对应被删掉的 DID 的读写回调函数声明, 和 27 服务变更无关, 属于 DID 列表精简的一致动作。

一句话结论

整份替换 Dcm_Lcfg 时, 这些连带变更不可避免。关键是要确认 P301 的 DaVinci 配置在以下几个点上与 C801 需求一致:

- 安全等级要求 (0x02 vs 0x03)
- mode rule 前置条件 (3u 对应的规则链)
- DID 列表是否多删少删了

如果确认无误, 这些变更都是合理的。

19服务和85服务都改了为什么

Thought for 4s >

Bash Search for 0x19 and 0x85 changes in the diff

```
IN grep -n "0x19\|Svc19\|DTC\|0x85\|Svc85\|ControlDTC" /home/c-ziming.jiang/.claude/projects/-home-c-ziming-jiang-catkin-ws-src-dsar-hq/80d94ac3-36ec-475a-8e9b-029abaa0fe2e/tool-results/beptdf6sq.t

OUT 20:@@ -247,9 +247,9 @@ CONST(uint8, DCM_CONST) Dcm_CfgSvc85SubFuncLookUpTable[3]=
216:-   ,{ Dcm_Service19Processor,0x01u, 1u,   5u,   0u, 2u, 2u} /* SID: 0x19 */
217:+   ,{ Dcm_Service19Processor,0x00u, 1u,   5u,   0u, 2u, 2u} /* SID: 0x19 */
---
```

Thought for 2s >

Read beptdf6sq.txt (lines 211-250)

Thought for 15s >

变更都在 Dcm_CfgDiagServiceInfo 和 Dcm_CfgDiagSvcIdExecPrecondTable 这两个数组里, 属于同一次替换的连锁反应:

0x19 (ReadDTCInformation)

字段	旧值 (C801原始)	新值 (P301)
安全等级	0x01 (需L1解锁)	0x00 (无需解锁)
执行前置条件	0u (DefaultModeRule)	2u

0x85 (ControlDTCSetting)

字段	旧值	新值
执行前置条件	0u (DefaultModeRule)	3u

0x27 (SecurityAccess)

字段	旧值	新值
安全等级	0x03	0x02
执行前置条件	0u	3u

为什么会一起变?

根本原因是这次提交是整份 Dcm_Lcfg 替换, 不是只改 27 服务的 seed/key 长度。P301 的 DaVinci 工程在 **mode rule (模式规则)** 的量级就和 C801 旧配置不同:

- C801 旧配置: 2 条 mode rule (Dcm_CfgModeMgrRules[2]), 前置条件只有 0u 和 1u
- P301 配置: 4 条 mode rule, 前置条件引入了 2u、3u

同时 Dcm_CfgStatePreconditions[] 也从 5 条变成了更多 (对应新增的 rule 索引)。

所以 19、85 服务的前置条件从 0u (放行) 变成 2u / 3u, 是 P301 的 DaVinci 对这 3 个服务统一加了更严格的执行限制, 数据上是连坐改变的。

建议

如果 C801 在 19 和 85 服务的行为没有特殊的合规/需求要求, 这些变更一般不会造成问题 (P301 早就在跑)。但还是要确认: C801 是否允许 19 服务在未解锁的情况下使用? 旧配置要求 L1 解锁才能读 DTC, 新配置删掉了这个限制。如果 C801 的安全策略不同意, 就需要回退这个字段。